The Field Journal
A PRACTICAL GUIDE TO AI SYSTEMS

Evaluate classification in the LLM era

Measure performance, establish credible reference labels, and understand what the metrics really mean.

THE DAILY CHRONICLE

The model is only part of the system

Labels, evaluation data, audits, and human-review policies all shape the reliability of a classifier.

I Establish the problem

We classify customer reviews to monitor sentiment, identify dissatisfied customers, and improve products and service.

1.1 Running example

Let's say we need to classify customer reviews as Positive, Neutral, or Negative. We want to identify dissatisfied customers and prioritize follow-up.

"The battery died after a week. Total waste of money."
Negative

Classification choice
Our example is **single-label**: one review receives one sentiment label. Other tasks may be multi-label, where one review can receive several labels.

Single-label *One review = one label*: Negative

Multi-label *One review = several labels*: Battery complaint; Delivery issue

1.2 Core evaluation metrics *(per class)*

For a class (e.g., Negative), measure how many predictions are correct and how many actual examples are detected.

$$\text{Precision} = \frac{TP}{TP + FP}$$
Of predicted examples, how many are correct?

$$\text{Recall} = \frac{TP}{TP + FN}$$
Of actual examples, how many are detected?

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$
Harmonic mean of precision and recall.

Macro F1: Compute F1 for each class (Positive, Neutral, Negative) and average equally.

1.3 Example results (100 reviews)

Confusion matrix: actual vs. predicted labels.

Actual class \ Predicted class	Pos	Neu	Neg	Total
Pos	65	4	1	70
Neu	5	13	2	20
Neg	7	2	1	10
Total	77	19	4	100

Accuracy = 79 / 100 = 0.790

The diagonal contains 79 correct predictions. The other cells reveal 21 errors, including 7 Negative reviews mistaken for **Positive.**

1.4 Negative-class metrics

Focus on the Negative class, where performance is weak.

$$\text{Precision} = \frac{1}{1+1+2} = \mathbf{0.250}$$
Of the 4 predicted Negative reviews, 1 is actually Negative.

$$\text{Recall} = \frac{1}{7+2+1} = \mathbf{0.100}$$
Of the 10 actual Negative reviews, only 1 was detected.

$F1 = \mathbf{0.143}$

Key point
The classifier misses 9 of 10 negative reviews, and 3 of 4 negative alerts are false alarms.

We can measure these errors because we have reference labels to compare against. | **But how do we know those labels are trustworthy?**

II Establish credible references

Use independent, human-reviewed labels to measure performance and guide reliable training data.

2.1 Classification metrics need labels: Where do they come from?

Classification metrics need reference labels. Human annotation is costly, LLMs can label more reviews, but they need validation.

Gold labels
- Human-reviewed annotations
- Higher credibility
- Smaller scale
- Created with an explicit rubric (often multiple reviewers)

Silver labels
- LLM-generated annotations
- Lower cost, larger scale
- Provisional labels
- Useful for training, but not treated as ground truth

Key point: Gold provides credible evaluation references, Silver provides scalable additional supervision — but isn't ground truth.

2.2. Example disagreement

LLM labels can be wrong, especially for nuanced language like sarcasm.

"Wonderful. Another battery that dies immediately."

Human (Gold): Negative ⟷ LLM (Silver): Positive

An incorrect silver label can become a training target and reinforce the model's mistakes.

Evaluation populations
We evaluate on two types of samples to understand different failure modes.

Representative sample
- Typical language and topics from expected use
- Estimates real-world performance

What happens in ordinary use?

Challenge sample
- Enriched with hard cases (e.g., sarcasm, negation, mixed sentiment)
- Reveals specific weaknesses

Where does the model struggle?

2.3 How much do human reviewers really agree?

Inter-rater agreement asks whether reviewers agree more than chance would predict.

A Cohen's κ • two reviewers

① What does it measure?
Two reviewers label the same reviews. Kappa asks whether their agreement exceeds what their individual label frequencies predict.

② Governing equation

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$
Actual agreement minus chance agreement, divided by the maximum improvement still possible beyond chance.

③ The 100-review example

A / B	Pos	Neg	Total
Pos	45	5	50
Neg	10	40	50
Total	55	45	100

④ Step-by-step calculation

1. **Observed agreement:** P_o

$$P_o = \frac{\text{matching labels}}{N}$$
Count agreements (45 + 40) and divide by all 100 reviews.

$$P_o = \frac{45+40}{100} = 0.850$$

2. **Chance agreement:** P_e

$$P_e = \sum_c p_{A,c}\, p_{B,c}$$
Multiply each category's marginal rates and add.

$$P_e = (50/100)(55/100) + (50/100)(45/100) = 0.500$$

3. **Correct for chance:** κ

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$
Plug in the observed and chance agreement.

$$\kappa = \frac{0.850 - 0.500}{1 - 0.500} = 0.700$$

Cohen's κ = 0.700

Interpretation: 70% of the possible improvement over chance. Agreement alone does not establish correct labels.

B Fleiss' κ • multiple reviewers

① What does it measure?
Several reviewers label the same reviews. Fleiss' κ compares their average within-review agreement with agreement expected from the pooled votes.

② Governing equation

$$\kappa_F = \frac{\bar{P} - P_e}{1 - P_e}$$
Average agreement across reviews, minus chance agreement, scaled by the remaining possible improvement.

③ The four-review example

Review	Pos	Neg	P_i
1	3	0	1.000
2	2	1	0.333
3	0	3	1.000
4	1	2	0.333

④ Step-by-step calculation

1. **Agreement within a review:** P_i

$$P_i = \frac{\sum_c n_{ic}(n_{ic} - 1)}{n(n-1)}$$
Counts the number of agreeing ordered reviewer pairs within review *i* (numerator) and divides by all possible ordered reviewer pairs (denominator).

$$P_2 = \frac{2(2-1) + 1(1-1)}{3(3-1)} = \frac{2}{6} = \frac{1}{3} = 0.333$$

2. **Mean observed agreement:** $\bar{P}$

$$\bar{P} = \frac{1}{N}\sum_i P_i$$
Average the four P_i values (1, 1/3, 1, 1/3).

$$\bar{P} = \frac{1 + 1/3 + 1 + 1/3}{4} = \frac{2}{3}$$
N = 4 reviews

3. **Chance-expected agreement:** P_e

$$p_c = \frac{\sum_i n_{ic}}{Nn}, \quad P_e = \sum_c p_c^2$$
Pooled category share p_c. Here, $p_{Pos} = 6/12$ and $p_{Neg} = 6/12$.

$$P_e = (6/12)^2 + (6/12)^2 = 0.500$$

4. **Correct for chance:** κ_F

$$\kappa_F = \frac{\bar{P} - P_e}{1 - P_e}$$
Substitute $\bar{P} = 2/3$ and $P_e = 0.5$.

$$\kappa_F = \frac{2/3 - 0.5}{1 - 0.5} = 0.333$$

Fleiss' κ = 0.333

Interpretation: modest agreement beyond chance, despite reviewers agreeing unanimously on two of the four reviews.

C Krippendorff's α • flexible ratings

① What does it measure?
Krippendorff measures reliability through disagreement. It can handle reviews with different numbers of valid ratings; here we use nominal labels.

② Governing equation

$$\alpha = 1 - \frac{D_o}{D_e}$$
Take actual disagreement as a fraction of expected disagreement, then subtract that fraction from one.

③ The four-review example

Review	R1	R2	R3
1	Pos	Pos	–
2	Neg	Pos	Neg
3	Pos	Pos	Pos
4	Neg	–	Neg

Ten valid ratings: 6 Positive, 4 Negative. Only Review 2 has mixed labels.

④ Step-by-step calculation

1. **Observed disagreement:** D_o

$$D_o = \frac{1}{n}\sum_i \frac{2a_i b_i}{m_i - 1}$$
Only Review 2 has mixed labels, so it contributes four ordered positive/negative disagreements; dividing by $(m_i - 1) = 2$ gives two weighted disagreement units, and dividing by $n = 10$ valid ratings gives $D_o = 0.200$.

$$D_o = \frac{1}{10}\left[0 + \frac{2(1)(2)}{3-1} + 0 + 0\right] = \frac{0+2+0+0}{10} = 0.200$$

2. **Expected disagreement:** D_e

$$D_e = \frac{2 n_{Pos} n_{Neg}}{n(n-1)}$$
The expected disagreement uses the pooled label counts and the finite-sample denominator $n(n-1) = 10 \times 9$, giving $48/90 \approx 0.533$.

$$D_e = \frac{2(6)(4)}{10(9)} = \frac{48}{90} \approx 0.533$$

3. **Correct for expectation:** α

$$\alpha = 1 - \frac{D_o}{D_e}$$
Use the observed and expected disagreement.

$$\alpha = 1 - \frac{0.200}{48/90} = 0.625$$

Krippendorff's α = 0.625

Interpretation: observed disagreement is 62.5% below the expected baseline. This does not establish label correctness.

Remember: Inter-rater agreement measures consistency, not whether the labels are correct.

2.4 Use gold labels to guide LLM labeling

Use a rubric plus a few gold examples to build one reusable prompt, then apply it consistently to many unlabeled reviews.

① Prompt template

Task: Classify the review as Positive, Neutral, or Negative.
Rubric: Focus on customer sentiment toward the product.
Gold examples:
- "Works perfectly." → Positive
- "Wonderful. Another battery dies immediately." → Negative

Now classify: {review}
Return JSON: one label field.

② Apply it to each review

```
For review in
unlabeled_reviews:
prompt = template.replace
    ("{review}", review)
 response = LLM(prompt)
 label = parse_label(response)
  save(review, label)
```

The same prompt is reused; only the review text changes.

③ Structured output

The model returns a label in a simple format, e.g.:

```
{"label": "Negative"}
```

Collect the returned labels as provisional silver labels.

Key point: Use gold examples to design the Teacher prompt. Refine it on development reviews, then independently audit the frozen recipe before labeling at scale.

2.5 Dataset composition and lineage

Two collections feed the labeling workflow: human-reviewed gold labels and a larger pool of unlabeled reviews. Each subset has a distinct role.

600 human-reviewed gold reviews
- Reviewed against an explicit rubric
- Used for prompt development, and final testing

10,000 initially unlabeled reviews
- Candidate reviews for LLM labeling
- A small sample is reserved for independent audit

200 training	200 development	200 final test (protected)	100 reserved for audit	9,900 candidates
Teacher examples and student training	Tune the Teacher prompt and compare candidate models	Final evaluation of the selected classification system	Independently human-reviewed to audit Teacher labels	Labeled with the finalized Teacher recipe to become provisional silver labels

Finalize Teacher recipe (few-shot examples + (development evaluation) → **Audit on 100 reserved** (human labels) → **Apply to 9,900 candidates** (provisional silver labels)

Key point: Keep development, audit, and final test sets separate. The 100-review audit set is used to evaluate the finalized labeling recipe — not to tune it.

III Build and compare classifiers

Now that we have credible labels, we can either classify directly with an LLM or use those labels to train a smaller classifier.

3.1 From labels to real-time inference

A new customer review arrives. We can either use the Teacher (an LLM) classify it directly, or deploy a smaller Student classifier trained on labeled reviews.

A. Teacher (LLM)—direct classification

- Classify each review directly with the LLM.
- No task-specific training required.
- Flexible and potentially high quality, but often higher cost and latency at scale.

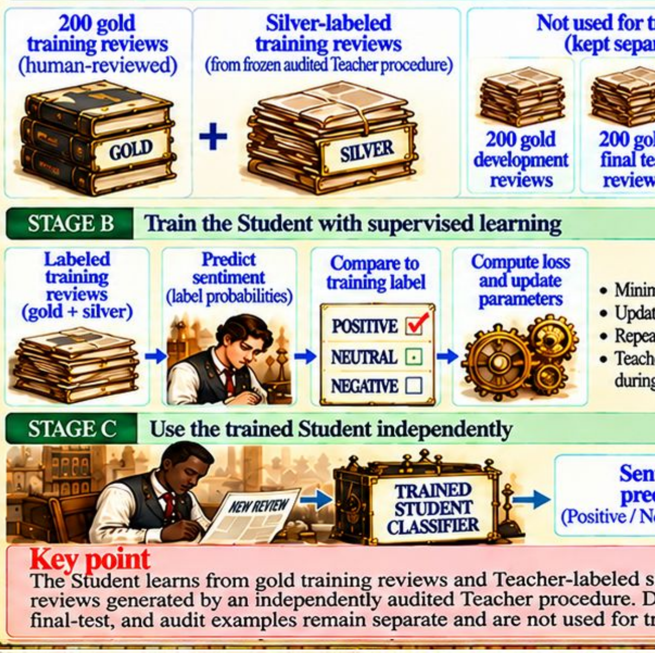
B. Student classifier—trained model

- Train a smaller, specialized model on labeled reviews.
- Lower inference cost and latency.
- Simpler deployment and output format.
- Can be tailored to the task and optimized for efficiency.

Key point
We can either use the Teacher (LLM) directly or train a smaller Student classifier. The best choice depends on the application, cost, latency, and deployment requirements.

3.2 How does the Student learn from labeled reviews?

The Student is trained using 200 gold training reviews plus silver-labeled training reviews generated by a frozen, independently audited Teacher labeling procedure. These labels are used for training, but not during inference.


STAGE A
Assemble training data
200 gold training reviews (human-reviewed)
GOLD
+
Silver-labeled training reviews (from frozen audited Teacher procedure)
SILVER
Not used for training (kept separate)
200 gold development reviews
200 gold final test reviews
Audit samples
STAGE B
Train the Student with supervised learning
Labeled training reviews (gold + silver)
Predict sentiment (label probabilities)
Compare to training label
POSITIVE ☑
NEUTRAL ☐
NEGATIVE ☐
Compute loss and update parameters
STAGE C
Use the trained Student independently
NEW REVIEW
TRAINED STUDENT CLASSIFIER
Sentiment prediction (Positive / Neutral / Negative)

- Minimize prediction error
- Update model parameters
- Repeat for many examples
- Teacher LLM is not used during training

Key point
The Student learns from gold training reviews and Teacher-labeled silver reviews generated by an independently audited Teacher procedure. Development, final-test, and audit examples remain separate and are not used for training.

3.3 Hypothetical example: when the Teacher gives the wrong label

A simplified example shows how an incorrect silver label can teach the Student the wrong relationship.

"Wonderful. Another battery that dies immediately."

Human reference (gold) (the true label): Negative
Teacher label (silver) (incorrect): Positive

① The Student currently predicts:

Class	Probability
Positive	0.30
Neutral	0.50
Negative	0.20

The Student's current prediction is Neutral (highest probability).

② Because the Teacher's target is Positive, the training update pushes the Student toward Positive — even though the human reference is Negative.

③ The update increases the model's tendency toward Positive, despite the Negative reference.

Key point
One bad label can mislead the model. Repeated errors can reinforce mistakes.

3.4 Which classifier should we deploy?

Compare using the same gold development set to select the best approach for the application. Then evaluate the selected system on the protected gold final test set.

Comparison criterion	Teacher (LLM) (direct inference)	Student (trained model)
Predictive quality (on gold development set)	Depends on task	Depends on task
Output format and flexibility	More flexible (generative)	Focused (class probabilities)
Inference latency (per review)	Often slower	Often faster
Cost per prediction	Often higher	Often lower
Deployment complexity	General-purpose service or API	Simpler, more controlled

Key point
Select the approach that best meets the application's performance and deployment requirements using the gold development set. Then evaluate the selected system on the protected final test set.

3.5 Does silver-labeled data help the Student?

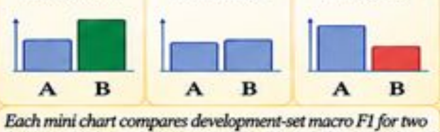
We train two identical Students: one on 200 human-gold examples, the other on the same gold examples plus 9,900 Teacher-labeled (silver) examples. Both are evaluated on a separate 200-example gold development set.

Training setups (student models)

Student A Gold only: Train on 200 gold training examples (human-reviewed).

Student B Gold + silver: Train on the same 200 gold examples plus 9,900 Teacher-labeled examples (machine-generated).

Possible outcomes (illustrative)
Development-set macro F1



Each mini chart compares development-set macro F1 for two Students: left bar = Student A (gold only), right bar = Student B (gold + silver). The question is whether adding silver improves performance on the independent gold development set.

Key point
More labeled data isn't necessarily better labeled data. Silver supervision helps only if it improves performance on independent human-gold examples. Otherwise, Teacher errors may outweigh the benefit of additional training data.

BRIDGE FROM III TO IV

Know what you're testing

A label depends on the model, prompt, settings, and input review. Repeat unchanged to test stability; vary one factor at a time to test sensitivity.

Model LLM and version (e.g., GPT-4o)
Prompt Instructions, label definitions, examples
Settings Temperature, top-p, max tokens, seed
Input Customer review (text to classify)

Experimental rule: Change only one factor at a time while holding the others fixed.

IV Stress-test the labeling procedure

How do we know a labeling procedure will remain reliable across repeated runs, harmless changes, and difficult reviews? | *Check stability, prompt sensitivity, and correctness against human references.*

A. Understand and measure instability *Identify when and why the procedure produces different labels.*

4.1 Why do labels vary?

The same input can produce different labels due to several factors.

Sampling variability	Randomness from temperature and decoding settings.
Prompt ambiguity	Instructions or rubric leave room for interpretation.
Input ambiguity	Review is mixed, nuanced, or sarcastic.
Boundary cases	Review sits between labels (e.g., Positive vs. Neutral).

Understanding the cause helps determine how to improve the procedure.

4.2 Repeated-run consistency

Run the same review through the same procedure multiple times. If the label changes, the procedure is unstable.

"Wonderful. Another battery that dies immediately."

- Run 1 → Positive
- Run 2 → Negative
- Run 3 → Positive
- Run 4 → Positive
- Run 5 → Negative

4.3 Measure consistency

Repeat the labeling multiple times and calculate agreement with the most frequent label (modal label).

Run 1	Run 2	Run 3	Run 4	Run 5
Pos	Neg	Pos	Pos	Neg

Modal-label frequency
3 / 5 = 60%

This measures stability (repeatability), not correctness. A higher agreement suggests a more stable procedure.

Three ways to examine consistency (classification adaptation of JHU)

Exact consistency	Same label recurs.
Correctness consistency	Label repeatedly matches human reference.
Failure consistency	Same incorrect label recurs.

4.4 Prompt sensitivity

Keep the review fixed, but slightly change the instructions. A robust procedure should usually keep the same label.

Prompt A Classify the customer's sentiment toward the product as Positive, Neutral, or Negative.

Prompt B Identify whether the customer expresses Positive, Neutral, or Negative sentiment toward the product.

"Wonderful. Another battery that dies immediately."

A → Negative B → Positive

Even small wording changes can sometimes flip the label.

4.5 Input perturbation

Keep the prompt fixed, but make small, meaning-preserving changes to the review (e.g., punctuation, wording, casing). The label should remain the same.

Review variation		Label
Original:	"Wonderful. Another battery that dies immediately."	Negative
Exclamation:	"Wonderful — another battery that dies immediately."	Negative
Synonym:	"Great, another battery that dies immediately."	Negative
Case change:	"WONDERFUL. Another battery that dies immediately."	Negative
Minor typo:	"Wonderful, Another battery that dies immediately."	Negative
Punctuation:	"Wonderful.... another battery that dies immediately."	Positive (correct flip)

Most variants preserve the label, but one harmless change flipped it, revealing a robustness failure on this review.

B. Interpret and improve *Use interventions to stabilize the procedure and understand what the outputs mean.*

4.6 Self-consistency (majority vote)

If repeated runs disagree, take the most frequent label. This can improve stability, but it does not guarantee correctness.

Run 1	Negative
Run 2	Negative
Run 3	Positive
Run 4	Negative

→ Majority vote **Negative** (3 of 4 runs)

Use voting to reduce randommess, but remember: all runs can be consistently wrong.

4.7 Consistency is not correctness

Stable outputs are useful, but stability (and confidence) alone is not enough. Compare the procedure against independent human references.

Review	Runs (labels)	Human reference	Interpretation
"Great battery life and good value."	Pos, Pos, Pos	Positive	Consistent and correct
"Wonderful. Another battery that dies immediately."	Pos, Pos, Pos	Negative	Consistent but wrong
"It works, I guess."	Pos, Neg, Pos	Neutral	Inconsistent

A label can be high confidence and still be wrong. Always check a sample against human references.

4.8 What does confidence tell us?

LLMs provide different confidence signals. None of them guarantees correctness.

Signal	What it measures	What it does not establish
Token log-probs	Model-assigned likelihoods of generated tokens.	Calibrated sentiment-class probabilities.
Self-reported confidence	The model's stated certainty (e.g., "I'm 90% sure").	A validated confidence estimate.
Repeated-run agreement	How often the same label recurs across independent runs.	That the agreed-upon label is correct.

Use confidence signals as diagnostic information. Validate them against known outcomes.

4.9 What should we change when a test fails?

Different failure patterns suggest different fixes. A single observation can have multiple causes.

Observed failure	Possible cause(s)	What to try next
Identical runs disagree	Sampling or vague instructions	Lower temperature, clarify rubric
Prompt variants flip label	Wording sensitive or unclear rubric	Rewrite instructions, add clearer examples.
Input perturbations flip label	Surface-form brittleness	Add diverse examples, improve examples.
Stable but wrong vs. human reference	Systematic misunderstanding	Revise prompt, check examples.
Human reviewers disagree	Ambigouus review or guidelines	Refine label definitions or allow abstain
Performance drops after change	Unintended side effect	Compare with prior version and revert

C. Establish trust *Freeze a reproducible recipe and validate it against an independent human-reviewed sample before generating silver labels at scale.*

4.10 Freeze and version the labeling recipe

A labeling procedure is more than one prompt. Treat it as a single recipe that must be versioned, so changes can be compared and rolled back.

Recipe components
- Task definition
- Label definitions
- Prompt text
- Few-shot examples
- Model and version
- Decoding settings
- Output schema
- Evaluation notes

Version history (example)

v1.0	Initial version
v1.1	Added clearer neutral definition and examples
v1.2	Revised examples and lowered temperature

Record changes, compare versions on the same development set, and revert if needed.

4.11 Finalize and audit against human references

Before using silver labels at scale, validate the frozen recipe on a reserved, human-reviewed audit sample.

Audit protocol (example)
1. Finalize the recipe using development examples.
2. Apply it to a reserved, human-reviewed sample. (e.g., 100 reviews).
3. Measure label correctness, repeated-run stability, and key failure patterns.
4. Decide whether the evidence supports using the procedure at scale, or if further revision is needed.

Ready for use when: (task-specific criteria)
- ✓ Stable across repeated runs
- ✓ Robust to prompt and input changes
- ✓ Acceptable agreement with human references
- ✓ Recipe is versioned and documented

Caveats:
1. Audit results apply only to the tested sample.
2. 100 examples may not cover all edge cases.
3. Do not use the audit sample to tune the recipe.
4. Audit fails? Revise on dev; re-audit fresh.

Key point: Before trusting silver labels at scale, verify that the labeling recipe is stable, robust, and checked against independent human references.

V Evaluate and deploy

Which failures remain, and which predictions should be automated? | *Measure final performance, understand linguistic failures, and deploy selectively.*

5.1 Final evaluation on a protected test set

Measure performance on unseen examples excluded from training, development, and audit.

200 protected gold test examples
- Excluded from training, development, and audit
- Evaluated once, after finalization
- Estimates out-of-sample performance

Example results (sentiment classification)

Metric	Positive	Neutral	Negative	Overall
Precision	0.80	0.73	0.80	--
Recall	0.77	0.69	0.85	--
F1 score	0.79	0.71	0.82	--
#examples	62	58	80	200
Overall accuracy	--	--	--	0.78 (156/200)

Interpreting results
- Report per-class metrics; overall accuracy can hide problems.
- With only 200 examples, performance estimates are uncertain.
- Never use test results to tune the model or threshold.

Key point: The protected test estimates performance on unseen examples from the population it represents.

5.2 Where classifiers struggle

Performance varies by language type. Some language features are harder to classify correctly than others.

Language type	Accuracy	#examples	Example
Straightforward	0.91	100	"Great product, works well."
Negation	0.75	40	"Not nearly as good as expected."
Mixed sentiment	0.65	40	"Brilliant camera, but battery issues."
Sarcasm	0.40	20	"Wonderful. It broke immediately."

Illustrative feature-based subsets, not a partition of the test set. Reviews may appear in multiple rows (e.g., negation and sarcasm).

Example errors on difficult cases (sarcasm, 20 examples)

Actual \ Predicted	Positive	Neutral	Negative	Total
Positive	3	1	1	5
Neutral	2	2	1	5
Negative	4	3	3	10
Total	9	6	5	20

8 of 20 correctly classified (40% accuracy). Many sarcastic Negatives are predicted Positive.

Key point: Examine difficult language separately to understand where and why the classifier makes mistakes.

5.3 Selective automation using confidence

Automate high-confidence predictions; route the rest to human review. Higher thresholds reduce coverage and may improve selective accuracy.

A. How it works
Use the highest predicted class probability:
- ≥ threshold: automate
- < threshold: review
- Label stays the same.

B. Example

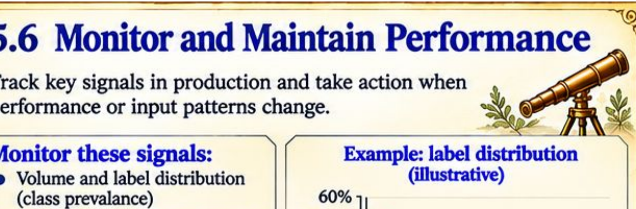
Predicted class:	Negative
Confidence:	0.80
Threshold ≥ 0.70 →	Automate
Threshold ≥ 0.90 →	Human review

⚠ Confidence is not necessarily calibrated: A score of 0.80 does not guarantee 80% correctness.

C. Illustrative operating-point results

Confidence threshold	% automated (coverage)	Accuracy (among automated)	Error rate (selective risk)
≥ 0.50	100%	78%	22%
≥ 0.70	88%	86%	14%
≥ 0.80	70%	90%	10%
≥ 0.90	45%	94%	6%

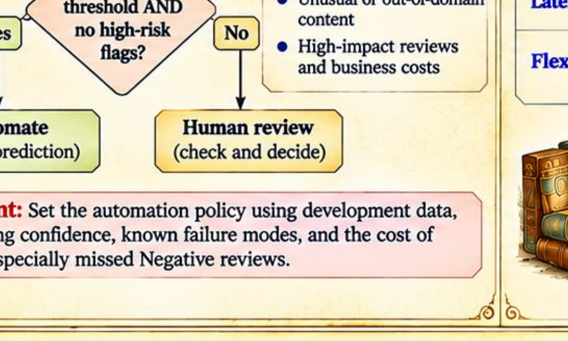
Accuracy and coverage as the threshold increases



Key point: Higher thresholds reduce coverage but can improve selective accuracy. Choose a threshold that matches your use case.

5.4 From Metrics to Decisions

Use development evidence to set when to automate or defer to human review. Evaluate the final policy on protected test data.


Classify review (get predicted class and confidence)
Confidence ≥ threshold AND no high-risk flags?
Yes
No
Automate (accept prediction)
Human review (check and decide)

High-risk flags (may override confidence):
- Difficult language (e.g., sarcasm, negation)
- Small margin between top classes
- Unusual or out-of-domain content
- High-impact reviews and business costs

Key point: Set the automation policy using development data, considering confidence, known failure modes, and the cost of errors—especially missed Negative reviews.

5.5 Choose What to Deploy

Select a deployment approach that balances task quality, compute cost, speed and flexibility for your needs.

Criterion	Teacher (large model)	Student (smaller model)
Task-specific quality	**Often higher** (especially on difficult language)	**Competitive on** narrower tasks, but can be lower.
Inference cost	**Higher** (more compute)	**Lower** (less compute)
Latency	**Slower** (longer response time)	**Faster** (shorter response time)
Flexibility	**Prompt-adjustable** (without retraining)	**Often requires retraining** (for learned changes)

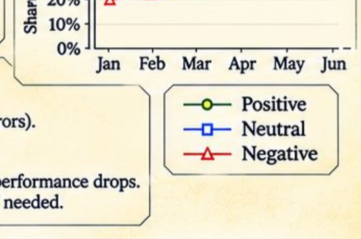
Common configurations:
- **Direct Teacher:** more flexible; often costlier.
- **Direct Student:** lower cost, faster; suitable for many cases.
- **Hybrid:** use the Student for routine reviews; escalate uncertain or high-risk cases to the Teacher or human review.

5.6 Monitor and Maintain Performance

Track key signals in production and take action when performance or input patterns change.

Monitor these signals:
- Volume and label distribution (class prevalance)
- Confidence and abstention rates (e.g., sent to human review)
- Human-reviewed errors (e.g., false positives/negatives)
- Changes in input language or topics (e.g., new products)

Example: label distribution (illustrative)



Take action when needed:
- Investigate unexpected changes (e.g., sudden shifts in labels or errors).
- Recalibrate or update the policy (e.g., adjust the threshold).
- Retrain with new labeled data if performance drops.
- Roll back to a previous version if needed.

Key point: Performance can drift as inputs and users change. Monitor, respond, and keep the system reliable over time.

Remember: Trustworthy classification depends on credible reference labels, independently evaluated performance, and deployment decisions that account for errors and uncertainty.

JHU / Great Learning Evaluating Generative AI Workflows | UMich MADS SIADS 542 Supervised Learning | DeepLearning.AI Week 3. Reinforcement Learning from Human Feedback | linkedin.com/in/jaimesingson